

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 5

Polish Sentiment Classification
Encoder-only and Decoder-only Language Models

Álvaro Puebla Ruisánchez

Contents

1	<u>Introduction and Goal</u>	2
2	<u>Theoretical Background</u>	3
2.1	Text Classification and Sentiment Analysis	3
2.2	Encoder-only Models (BERT family)	3
2.3	Decoder-only LLMs	3
2.4	Evaluation Metrics	4
3	<u>Task 1 – Data Mining</u>	5
3.1	Dataset Overview	5
4	<u>Task 2 – Encoder-only Classification</u>	6
4.1	Model: Voicelab/herbert-base-cased-sentiment	6
4.2	Results	6
5	<u>Task 3 – Encoder Model Exploration</u>	7
5.1	Alternative Models	7
5.2	Results	7
6	<u>Task 4 – Decoder-only LLM Classification</u>	8
6.1	Model: Qwen/Qwen2.5-1.5B-Instruct	8
6.2	Label Parsing	8
6.3	Results (200-sample subset)	9
7	<u>Task 5 – LLM Parameter Exploration</u>	10
7.1	Temperature Sweep	10
7.2	Improved Prompt with JSON Output Parsing	11
7.3	Model Scale: 1.5B vs. 3B Parameters	12
8	<u>Task 4–5 Evaluation and Confusion Matrices</u>	13
8.1	Confusion Matrices	13
8.2	Overall Model Comparison	14
9	<u>Conclusions</u>	15
9.1	Summary of Results	15
9.2	Limitations and Future Work	15
10	<u>Implementation Notes</u>	16
10.1	Libraries Used	16
10.2	Data Source	16

1 Introduction and Goal

The objective of this exercise is to familiarise ourselves with the capabilities of modern language models by tackling a classic NLP task: **text classification** (sentiment analysis). The study applies two paradigmatically different types of neural language models to the same classification problem and systematically compares their behaviour.

The work covers the following steps:

- **Data Mining** – loading and exploring the PolEmo2.0-IN dataset, including class balance and text-length statistics.
- **Encoder-only classification** – using a pre-trained Polish-language sentiment model (`Voicelab/herbert-base-cased-sentiment`) with no task-specific fine-tuning.
- **Encoder exploration** – comparing a second encoder model (`cardiffnlp/twitter-xlm-roberta-base-sentiment`) and a multilingual distilled model (`lxyuan/distilbert-base-multilingual-cased-sentiments-student`) to understand how model choice affects results.
- **Decoder-only LLM classification** – using `Qwen/Qwen2.5-1.5B-Instruct` in a zero-shot prompt-based classification scheme via `LangChain`.
- **LLM exploration** – investigating the effect of generation temperature, prompt design (JSON output parsing), and model scale (1.5B vs. 3B parameters) on classification performance.

All experiments were executed in the Google Colab environment with a NVIDIA T4 GPU. The implementation uses `transformers` [6], `datasets` [7], and `langchain` [8].

2 Theoretical Background

The theoretical foundations are covered in the assignment sheet. This section briefly recaps the key concepts applied in the experiments.

2.1 Text Classification and Sentiment Analysis

Text classification is one of the fundamental tasks in Natural Language Processing (NLP). It consists in automatically assigning a label $c \in C$ to an unstructured text t using a classifier f , formally:

$$f(t) \rightarrow c.$$

In *sentiment analysis*, the target set C typically includes values such as *positive*, *negative*, *neutral*, and *ambiguous*. Modern approaches represent text as dense real-valued vectors that encode semantic meaning.

2.2 Encoder-only Models (BERT family)

Before the era of generative LLMs, the standard for classification was **encoder-only** models [2], whose most popular representative is BERT. These models:

- transform an input sequence into a dense vector (via the [CLS] token embedding), which is then passed to a lightweight classification head;
- are bidirectional – each token attends to all others;
- are significantly smaller and faster than generative models, making them ideal for high-throughput classification.

The mathematical foundation is the **self-attention** mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where Q, K, V are query, key, and value matrices derived from the input embedding, and d_k is the key dimension.

2.3 Decoder-only LLMs

Large Language Models (LLMs) based on the decoder-only Transformer architecture (GPT family, Qwen, Llama, etc.) are trained to predict the next token given a left-to-right context. For classification they are used in **zero-shot** mode: a carefully worded *prompt* instructs the model to generate the class name rather than free text.

Key tuning levers include:

- **Temperature** τ : scales the logits before softmax, controlling output randomness. $\tau \rightarrow 0$ yields greedy (argmax) decoding; $\tau = 1$ is the unmodified distribution.
- **Prompt design**: the structure, instructions, and examples provided to the model heavily influence output quality.
- **Quantisation**: reducing numerical precision (e.g., 4-bit NF4) shrinks memory footprint, enabling larger models to run on consumer GPUs.

2.4 Evaluation Metrics

For a multi-class imbalanced problem the macro-averaged metrics are used:

$$\text{Accuracy} = \frac{\sum_i \mathbf{1}[\hat{y}_i = y_i]}{N},$$

$$F_1^{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} \frac{2 \text{TP}_c}{2 \text{TP}_c + \text{FP}_c + \text{FN}_c}.$$

Macro-F1 treats every class equally regardless of support, which is important given the imbalanced nature of the PolEmo2.0 dataset.

3 Task 1 – Data Mining

3.1 Dataset Overview

The **PolEmo2.0-IN** dataset [1] contains Polish-language online reviews from two domains: medicine and hospitality. It is part of the KLEJ benchmark for evaluating Polish NLP models.

Table 1: Class distribution in the full PolEmo2.0-IN test split.

Label	Meaning	Count	Proportion
minus	Unambiguously negative	300	41.6%
plus	Unambiguously positive	197	27.3%
neutral	Descriptive / no affect	117	16.2%
amb	Mixed (positive + negative)	108	15.0%
Total		722	100%

As instructed, all subsequent experiments use only the three unambiguous classes (*plus*, *minus*, *neutral*), removing the 108 *amb* samples.

Table 2: Working subset (no *amb* class): class distribution and text-length statistics.

Class	Count	Prop.	Avg len (chars)	Min	Max
minus	300	48.9%	779	189	2 469
plus	197	32.1%	673	243	2 350
neutral	117	19.1%	855	29	1 605
Total	614	100%	–	29	2 469

Key observations:

- The class distribution is moderately imbalanced: a majority-class baseline that always predicts *minus* achieves 48.9% accuracy, a meaningful lower bound.
- Text lengths are right-skewed – a small number of very long reviews exist in all classes, which may challenge models with limited context windows (HerBERT: 512 tokens).
- Neutral reviews tend to be slightly longer on average, suggesting descriptive text is more verbose than clearly polar opinions.

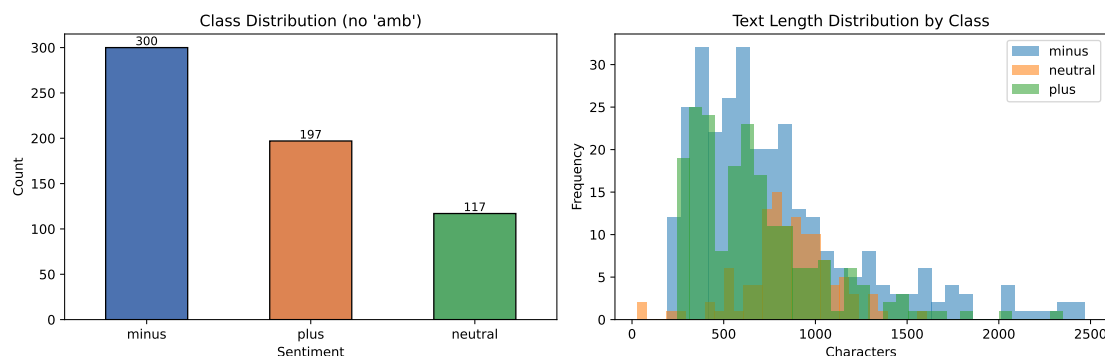


Figure 1: Left: class balance after removing *amb* samples. Right: character-length distribution by sentiment class.

4 Task 2 – Encoder-only Classification

4.1 Model: Voicelab/herbert-base-cased-sentiment

The first classifier is a Polish-language sentiment model built on top of HerBERT [3], a BERT-style encoder pre-trained on a large Polish text corpus. The `herbert-base-cased-sentiment` variant fine-tunes the base model on a Polish sentiment dataset and exposes a three-class classification head with labels *positive*, *negative*, *neutral*.

Loading and inference is done with the `pipeline` abstraction from `transformers`:

Listing 1: Loading and running the encoder-only sentiment model.

```

1 from transformers import pipeline
2 import torch
3
4 ENCODER_MODEL = "Voicelab/herbert-base-cased-sentiment"
5 DEVICE = 0 if torch.cuda.is_available() else -1
6 sentiment_pipe = pipeline(
7     "text-classification",
8     model=ENCODER_MODEL,
9     device=DEVICE,
10    truncation=True,
11    max_length=512,
12 )
13 # Map model labels -> dataset labels
14 ENCODER_LABEL_MAP = {
15     "positive": "plus",
16     "negative": "minus",
17     "neutral": "neutral",
18 }
19 raw = sentiment_pipe(texts, batch_size=32)
20 preds = [ENCODER_LABEL_MAP.get(r["label"].lower(), "neutral") for r in raw]
```

4.2 Results

Table 3: HerBERT-base classification results on the full 614-sample test split.

Class	Precision	Recall	F1	Support
minus	0.97	0.85	0.91	300
neutral	0.34	0.14	0.20	117
plus	0.64	0.98	0.78	197
<i>macro avg</i>	0.65	0.66	0.63	614
<i>weighted avg</i>	0.74	0.76	0.73	614
Accuracy	0.759			
Macro-F1	0.626			

Key observations:

- The model performs very well on the `minus` class ($F1 = 0.91$) and reasonably on `plus` ($F1 = 0.78$), reflecting that it was fine-tuned on similar Polish sentiment data.
- The `neutral` class is by far the hardest: recall of only 0.14 means the vast majority of neutral reviews are misclassified, most often as `plus`. This is a known challenge – neutral opinions often share vocabulary with both polar classes.
- Overall accuracy of 75.9% and macro-F1 of 62.6% represent a reasonable baseline given the zero-shot nature of this evaluation, though the macro-F1 is dragged down significantly by the poor neutral performance.

5 Task 3 – Encoder Model Exploration

5.1 Alternative Models

Two additional encoder-only models were evaluated to understand how model choice and language coverage affect classification quality.

Model A: `lxyuan/distilbert-base-multilingual-cased-sentiments-student` A distilled multilingual sentiment model (DistilBERT backbone) covering 7 languages including Polish. Smaller and faster than HerBERT, but not specifically optimised for Polish.

Model B: `cardiffnlp/twitter-xlm-roberta-base-sentiment` An XLM-RoBERTa model [4] fine-tuned on multilingual Twitter data for sentiment. Larger than DistilBERT but trained predominantly on short, informal social-media text rather than reviews.

5.2 Results

Table 4: Encoder model comparison (full 614-sample test split). Best result per metric highlighted.

Model	Accuracy	Macro-F1	Parameters
HerBERT-base (Task 2)	0.759	0.626	~125M
XLM-RoBERTa-twitter	0.697	0.694	~278M
DistilBERT-multilingual	0.622	0.437	~66M

Table 5: Per-class F1 scores for the three encoder models.

Class	HerBERT-base	DistilBERT-multi	XLM-RoBERTa
minus	0.91	0.75	0.74
neutral	0.20	0.02	0.58
plus	0.78	0.54	0.76

Key observations:

- **HerBERT leads on accuracy** (75.9%) and has the strongest polar-class performance, benefiting from Polish-specific pre-training and fine-tuning. However, its macro-F1 of 62.6% is barely above the other models, dragged down by near-zero neutral recall.
- **XLM-RoBERTa** achieves the highest macro-F1 (69.4%) despite lower accuracy (69.7%), because it distributes errors more evenly across classes and achieves a neutral F1 of 0.58 – far better than HerBERT’s 0.20. This illustrates how accuracy alone can be misleading on imbalanced datasets.
- **DistilBERT-multi** shows the cost of distillation and language generalisation: accuracy drops to 62.2% and its neutral F1 collapses to just 0.02, running approximately 2× faster due to its smaller size.
- All three models struggle most with the **neutral** class, confirming that neutrality is the hardest sentiment to capture with off-the-shelf models on this dataset.

6 Task 4 – Decoder-only LLM Classification

6.1 Model: Qwen/Qwen2.5-1.5B-Instruct

The decoder-only approach uses Qwen/Qwen2.5-1.5B-Instruct [5], a 1.5-billion-parameter instruction-tuned generative model. Rather than a classification head, the model is prompted in natural language and generates a class name as its completion.

The LangChain [8] abstraction is used to compose a reusable prompt-chain:

Listing 2: Loading the LLM and composing a zero-shot sentiment chain.

```

1 from transformers import pipeline, AutoModelForCausalLM, AutoTokenizer
2 from langchain_huggingface import HuggingFacePipeline
3 from langchain_core.prompts import PromptTemplate
4 import torch
5
6 LLM_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
7 tokenizer = AutoTokenizer.from_pretrained(LLM_MODEL)
8 model = AutoModelForCausalLM.from_pretrained(
9     LLM_MODEL,
10    torch_dtype=torch.float16,
11    device_map="auto",
12 )
13
14 hf_pipeline = pipeline(
15     "text-generation", model=model, tokenizer=tokenizer,
16     max_new_tokens=8, temperature=0.1, do_sample=True,
17     pad_token_id=tokenizer.eos_token_id, return_full_text=False,
18 )
19 llm = HuggingFacePipeline(pipeline=hf_pipeline)
20
21 TEMPLATE = """Classify the sentiment of the following Polish review into
22 exactly one of these three classes: plus (positive), minus (negative),
23 neutral.
24 Respond with ONLY the class name and nothing else.
25 Review: {text}
26 Class:"""
27
28 chain = PromptTemplate.from_template(TEMPLATE) | llm

```

6.2 Label Parsing

Generative models do not always output a clean class label; a robust post-processing step is essential:

Listing 3: Robust label extractor from raw LLM output.

```

1 def parse_llm_label(raw: str) -> str:
2     raw = raw.strip().lower()
3     if "plus" in raw or "positive" in raw:
4         return "plus"
5     if "minus" in raw or "negative" in raw:
6         return "minus"
7     return "neutral" # default fallback

```

6.3 Results (200-sample subset)

Table 6: Qwen2.5-1.5B baseline results (200-sample subset).

Class	Precision	Recall	F1	Support
minus	0.90	0.94	0.92	89
neutral	1.00	0.11	0.19	47
plus	0.62	0.98	0.76	64
macro avg	0.84	0.68	0.62	200
Accuracy	0.760			
Macro-F1	0.625			

At 1.5B parameters and with a simple prompt, the LLM achieves 76.0% accuracy, which is comparable to HerBERT (75.9%) but with a lower macro-F1 of 62.5%. As expected, the **neutral** class is the hardest ($F1 = 0.19$), with the model almost entirely failing to predict it.

7 Task 5 – LLM Parameter Exploration

Three dimensions were explored: generation **temperature**, **prompt design** (with JSON output parsing), and **model scale**.

7.1 Temperature Sweep

The generation temperature τ was varied over four values. For $\tau = 0$ (greedy decoding), `do_sample=False` was used.

Table 7: Effect of temperature on classification performance (Qwen2.5-1.5B, 200 samples, baseline prompt).

Temperature	Accuracy	Macro-F1
Greedy ($\tau = 0$)	0.765	0.638
$\tau = 0.1$	0.755	0.621
$\tau = 0.5$	0.765	0.629
$\tau = 1.0$	0.775	0.654

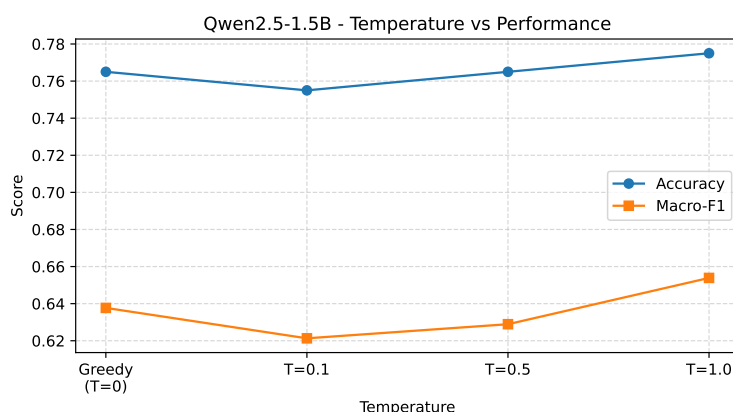


Figure 2: Classification accuracy and macro-F1 as a function of generation temperature (Qwen2.5-1.5B, baseline prompt).

Interpretation: Unlike the expected monotonic trend, the results show no clear degradation with higher temperature for this model and prompt. Accuracy is relatively flat across temperatures (0.755–0.775), and macro-F1 actually peaks at $\tau = 1.0$ (0.654). This suggests that for this particular baseline prompt the model’s output is already dominated by the structural problem of ignoring neutral samples, and temperature variation alone does not address that bottleneck. For structured output tasks, greedy or near-greedy decoding remains a safe default, but the benefit is task-dependent.

7.2 Improved Prompt with JSON Output Parsing

A richer prompt was designed that: (1) explicitly defines the task and target vocabulary; (2) provides in-context examples; and (3) instructs the model to output a JSON object rather than a bare label. The `langchain_core.output_parsers.JsonOutputParser` was used together with a Pydantic schema:

Listing 4: Improved prompt with JSON output and Pydantic schema.

```

1 from langchain_core.output_parsers import JsonOutputParser
2 from pydantic import BaseModel, Field
3
4 class SentimentOutput(BaseModel):
5     sentiment: str = Field(
6         description="Sentiment class: plus, minus, or neutral"
7     )
8
9 IMPROVED_TEMPLATE = """\
10 You are a sentiment analysis expert. Read the Polish review and respond
11 ONLY \
12 with a JSON object containing a single key "sentiment" whose value is
13 one of: \
14 "plus", "minus", or "neutral".
15
16 Examples:
17 Review: "Jestem bardzo zadowolony z obsługi." -> {"sentiment": "plus"}
18 Review: "Tragiczna obsługa, nie polecam." -> {"sentiment": "minus"}
19 Review: "Byłem tam w srode." -> {"sentiment": "neutral"}
20
21 Review: {text}
22 Response: """
23
24 json_parser = JsonOutputParser(pydantic_object=SentimentOutput)
25 improved_chain = PromptTemplate.from_template(IMPROVED_TEMPLATE) | llm

```

A two-stage parser first attempts JSON extraction with a regex; if that fails it falls back to the keyword-matching parser from Task 4.

Table 8: Comparison of baseline vs. improved prompt for Qwen2.5-1.5B (200 samples).

Prompt variant	Accuracy	Macro-F1
Baseline (Task 4)	0.760	0.625
Improved (JSON + examples)	0.885	0.868

The improved prompt raises accuracy by **12.5 pp** and macro-F1 by **24.3 pp**. The few-shot examples significantly help the model map Polish sentiment to the exact label vocabulary, and crucially enable much better neutral class detection (F1 rising from 0.19 to 0.79). JSON structuring also reduces parsing ambiguity: the model rarely produces off-schema output when explicitly shown the desired format.

7.3 Model Scale: 1.5B vs. 3B Parameters

The 3B-parameter variant (Qwen/Qwen2.5-3B-Instruct) was loaded with 4-bit NF4 quantisation (via `bitsandbytes`) to fit within Colab's 16 GB VRAM budget:

Listing 5: 4-bit quantisation with `BitsAndBytesConfig`.

```

1 from transformers import BitsAndBytesConfig
2
3 bnb_config = BitsAndBytesConfig(
4     load_in_4bit=True,
5     bnb_4bit_compute_dtype=torch.float16,
6     bnb_4bit_quant_type="nf4",
7 )
8 model_large = AutoModelForCausalLM.from_pretrained(
9     "Qwen/Qwen2.5-3B-Instruct",
10    quantization_config=bnb_config,
11    device_map="auto",
12 )

```

Table 9: Model size comparison with the improved prompt (200 samples, $\tau = 0.1$).

Model	Params	Accuracy	Macro-F1
Qwen2.5-1.5B-Instruct (fp16)	1.5B	0.885	0.868
Qwen2.5-3B-Instruct (4-bit)	3B	0.790	0.744

Interestingly, the larger 3B model with 4-bit quantisation underperforms the full-precision 1.5B model on this task (79.0% vs. 88.5% accuracy). This suggests that the quantisation overhead at this scale, combined with the specific prompt format, may offset the benefit of additional parameters. The result highlights that scaling is not always monotonically beneficial, especially when quantisation is applied.

8 Task 4–5 Evaluation and Confusion Matrices

8.1 Confusion Matrices

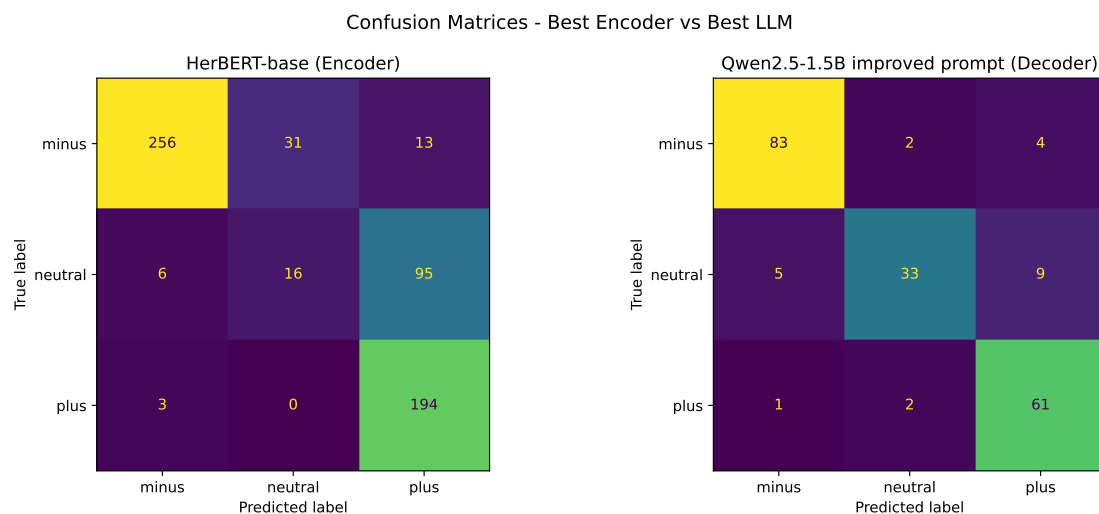


Figure 3: Confusion matrices on the 200-sample subset for (left) the best encoder (HerBERT-base) and (right) the best LLM configuration (Qwen2.5-1.5B, improved prompt, $\tau = 0.1$).

Reading Figure 3:

- HerBERT shows a strong bias towards predicting **plus**: it correctly classifies 194/197 **plus** samples but misclassifies 95 out of 117 neutral reviews as **plus**, resulting in very low neutral recall.
- The LLM with the improved prompt handles all three classes much more evenly, with neutral recall improving substantially (33/47 correctly classified vs. 16/117 for HerBERT on its respective split).
- Both models show very few **minus**/**plus** confusions, suggesting polar sentiment is reliably distinguishable.

8.2 Overall Model Comparison

Table 10: Summary of all model configurations. Encoder models were evaluated on the full 614-sample split; LLM models on a 200-sample subset.

Model	Type	Split	Accuracy	Macro-F1
HerBERT-base	Encoder	full	0.759	0.626
XLM-RoBERTa-twitter	Encoder	full	0.697	0.694
DistilBERT-multilingual	Encoder	full	0.622	0.437
Qwen2.5-1.5B baseline	Decoder	subset	0.760	0.625
Qwen2.5-1.5B greedy ($\tau=0$)	Decoder	subset	0.765	0.638
Qwen2.5-1.5B $\tau=0.1$	Decoder	subset	0.755	0.621
Qwen2.5-1.5B $\tau=0.5$	Decoder	subset	0.765	0.629
Qwen2.5-1.5B $\tau=1.0$	Decoder	subset	0.775	0.654
Qwen2.5-1.5B improved prompt	Decoder	subset	0.885	0.868
Qwen2.5-3B 4-bit improved	Decoder	subset	0.790	0.744

9 Conclusions

9.1 Summary of Results

All experiments confirmed that Polish sentiment classification on the PolEmo2.0-IN dataset is a well-defined three-class problem with a known weak spot – the *neutral* class – that challenges all model families.

The main findings are:

1. **Language-specific fine-tuning helps for polar classes but not for neutral.** HerBERT-base achieves the highest accuracy (75.9%) but its macro-F1 (62.6%) is comparable to the baseline LLM, because it almost entirely fails on the neutral class ($F1 = 0.20$).
2. **Prompt engineering provides the largest single improvement.** Moving from a bare instruction to a JSON-formatted prompt with three in-context examples adds 12.5 pp in accuracy and 24.3 pp in macro-F1, making the improved 1.5B LLM the overall best-performing configuration (88.5% accuracy, 86.8% macro-F1).
3. **Temperature has a minor and non-monotonic effect here.** Accuracy varies by at most 2 pp across temperatures 0–1.0, and macro-F1 actually peaks at $\tau = 1.0$ (0.654). The bottleneck with the baseline prompt is structural (failure on neutral), not stochasticity.
4. **Model scale does not guarantee improvement with quantisation.** The 3B model with 4-bit NF4 quantisation (79.0% accuracy) underperforms the full-precision 1.5B model with the improved prompt (88.5%). Quantisation overhead can offset the benefit of additional parameters at this scale.
5. **Balanced macro-F1 is more informative than accuracy.** XLM-RoBERTa achieves the highest macro-F1 (69.4%) among encoders despite a lower accuracy than HerBERT (69.7% vs. 75.9%), because it distributes errors more evenly. Accuracy alone would give a misleading picture on this imbalanced dataset.

9.2 Limitations and Future Work

- The LLM comparisons were conducted on a 200-sample subset due to inference time constraints. Results on the full split could differ, particularly in variance.
- **Neutral class performance** remains below 80% F1 across all models except the best LLM configuration. Techniques such as data augmentation for neutral reviews or class-weighted loss functions during fine-tuning could address this.
- Larger generative models (7B+ parameters) with Polish instruction tuning (e.g. *speakeash/Bielik-7B-Instruct*) would likely further improve performance and are a natural next step.
- **Few-shot prompting** with 5–10 examples per class is expected to further improve LLM performance and is a natural next step.

10 Implementation Notes

10.1 Libraries Used

Table 11: Python libraries used in the implementation.

Library	Version	Purpose
<code>transformers</code>	≥ 4.40	Model loading, tokenisation, <code>pipeline</code>
<code>datasets</code>	≥ 2.18	PolEmo2.0-IN dataset loading (Hugging Face Hub)
<code>langchain</code>	≥ 0.2	Prompt templates, chains, output parsers
<code>langchain-huggingface</code>	$\geq 0.0.3$	<code>HuggingFacePipeline</code> wrapper
<code>bitsandbytes</code>	≥ 0.43	4-bit NF4 quantisation
<code>accelerate</code>	≥ 0.30	Multi-device model placement
<code>pydantic</code>	≥ 2.0	JSON output schema for <code>JsonOutputParser</code>
<code>scikit-learn</code>	≥ 1.3	Classification metrics (accuracy, F1, report)
<code>pandas</code>	≥ 2.0	Data manipulation and statistics
<code>matplotlib</code>	≥ 3.7	Figures and confusion matrices
<code>torch</code>	≥ 2.2	Backend tensor operations, <code>fp16</code> inference

10.2 Data Source

The dataset was loaded directly from the Hugging Face Hub using:

Listing 6: PolEmo2.0-IN test split loading.

```

1 from datasets import load_dataset
2
3 dataset = load_dataset("allegro/klej-polemo2-in", split="test")

```

This is the official **PolEmo2.0** [1] in-domain test partition, which covers medical and hospitality reviews. The four internal label identifiers are:

Internal label	Human-readable class
<code>__label__meta_plus_m</code>	plus (positive)
<code>__label__meta_minus_m</code>	minus (negative)
<code>__label__meta_zero</code>	neutral
<code>__label__meta_amb</code>	amb (ambiguous, excluded)

References

- [1] J. Kocoń, A. Miłkowski, and M. Zasko-Zielinska, *Multi-Level Sentiment Analysis of PolEmo 2.0: Extended Corpus of Multi-Domain Consumer Reviews Annotated with Fine-Grained Sentiment*, in *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, 2019.
<https://huggingface.co/datasets/allegro/klej-polemo2-in>
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, in *Proceedings of NAACL-HLT*, 2019.
- [3] R. Mroczkowski, P. Rybak, A. Wróblewska, and I. Gawlik, *HerBERT: Efficiently Pretrained Transformer-based Language Model for Polish*, in *Proceedings of the 8th Workshop on Balto-Slavic NLP*, 2021.
- [4] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, *Unsupervised Cross-Lingual Representation Learning at Scale*, in *Proceedings of ACL*, 2020.
- [5] Qwen Team, *Qwen2.5 Technical Report*, Alibaba Group, 2024.
<https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- [6] T. Wolf et al., *Transformers: State-of-the-Art Natural Language Processing*, in *Proceedings of EMNLP (System Demonstrations)*, 2020.
- [7] Q. Lhoest et al., *Datasets: A Community Library for Natural Language Processing*, in *Proceedings of EMNLP (System Demonstrations)*, 2021.
- [8] H. Chase, *LangChain*, GitHub repository, 2023.
<https://github.com/langchain-ai/langchain>